SE-JEPA: Self-Evolving Latent World Models

Anonymous CVPR submission

Abstract

Latent world models let agents plan by imagining the outcomes of their actions, but they stop learning once deployed: when objects or appearances shift, prediction errors become planning failures. Test-time adaptation such as AdaJEPA repairs these errors within an episode but then resets, meeting every recurring shift as if for the first time. We therefore seek a world model that remembers what it learns. However, we find that simply keeping the adapted weights fails: updates across episodes are nearly orthogonal, and accumulating them degrades planning. To overcome this, we present SE-JEPA, a self-evolving latent world model. Inspired by complementary learning systems, it pairs fast within-episode adaptation with slow cross-episode consolidation, replaying experience into slow learners, one per self-discovered domain. Supervised only by its own prediction errors, SE-JEPA decides what to consolidate, where to store it, and when to trust it for planning. Because its pretrained model is never overwritten, nothing it already knows can be forgotten. It requires no shift labels or task boundaries and benefits both gradient-based and sampling-based planners. Across unseen object shapes, visual shifts, and deformable rope and granular manipulation, SE-JEPA grows stronger the longer it is deployed, setting a new state of the art by a wide margin.

1. Introduction

An agent using a visual world model chooses actions by predicting where they will leave an object and comparing that prediction with an image goal [22, 36, 41]. An unfamiliar shape can change how the object responds to a push. A change in appearance can alter how the model encodes the same motion. In either case, a controller that relies on the pretrained predictor can choose actions that miss the goal.

An executed push reveals how the scene actually changed. AdaJEPA [35] uses this observation to update the predictor before the controller replans. At the next episode, it resets the predictor. If the same object returns with a different goal, the agent has to learn its dynamics again.

Can an agent reuse what it learned on the previous encounter? A natural approach is to keep the adapted weights.

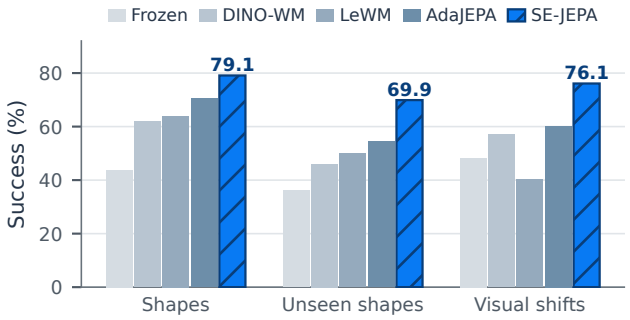


Figure 1. **Deployment experience improves held-out planning.** Held-out success with CEM planning on PushObj shapes, the three shapes unseen during pretraining, and six PushT visual shifts.

Yet each update fits the trajectory between one start and one goal. On a stream of pushes with matched targets, successive updates point in nearly orthogonal directions, and carrying all of them forward reduces success relative to resetting the predictor (Sec. 3.1). The executed trajectories give us another way to retain experience. We can replay pushes from several encounters and fit a predictor that must explain them together.

We introduce SE-JEPA, a latent world model that pairs within-episode adaptation with slower learning across episodes, inspired by complementary learning systems [23]. The pretrained predictor identifies trajectories it predicts poorly. SE-JEPA stores those trajectories in the pretrained encoder's latent space and groups them by fast-update direction and predictive compatibility. Each group trains a slow predictor by replay. A predictor becomes available for planning once it improves on the pretrained reference on incoming trajectories evaluated before training.

We evaluate whether this memory helps an agent reach new goals. On PushObj and PushT, SE-JEPA improves held-out success with both gradient descent (GD) and the cross-entropy method (CEM), while using fewer replans than episodic adaptation (Fig. 1). Beyond these task-trained ResNet models, replay also improves a DINO-WM predictor on rope and granular manipulation. A rope predictor trained during a single pass transfers to previously unseen instances.

Our contributions are threefold.

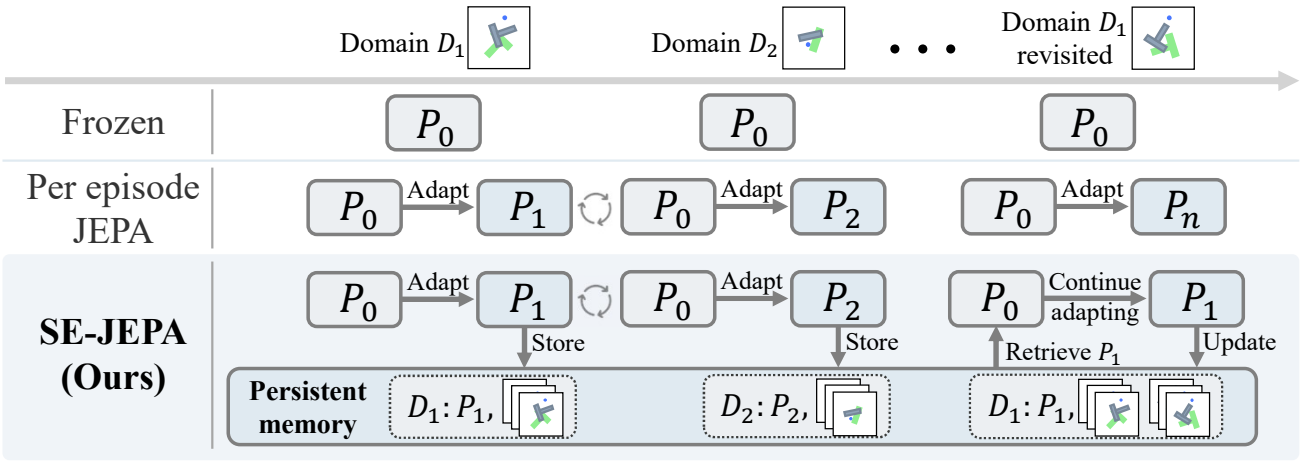


Figure 2. **From episodic adaptation to self-evolution.** A frozen world model plans every episode with the same predictor. Single-episode adaptation repairs the predictor during an episode and resets it afterwards. SE-JEPA consolidates experience into slow learners by replay and retrieves them when a shift recurs, so experience from earlier episodes can improve later planning. Domain names are for illustration; the method receives no labels.

- A matched-stream experiment measures the planning cost of carrying fast updates across episodes and shows how their directions vary from one encounter to the next.
- SE-JEPA discovers which experiences to replay together and tests each resulting predictor on incoming trajectories before using it for control.
- Our evaluation separates gains during deployment from transfer to unseen targets. It tests task-trained ResNet models on rigid objects and a DINO-WM backbone on deformable objects.

2. Related Work

World models for control. Learned dynamics support control by predicting the consequences of candidate actions. Embed to Control [37] learns locally linear latent dynamics, World Models [8] learns recurrent simulators, and PETS [6] combines probabilistic ensembles with planning. PlaNet [9] plans in latent space; Dreamer and its successors learn behavior through latent imagination [10–12]. SimPLe [16] and IRIS [24] learn visual simulators for data-efficient policy learning. MuZero [29] and EfficientZero [38] learn models tailored to search, while TD-MPC and TD-MPC2 [13, 14] combine latent dynamics with value learning for continuous control. We study how a visual planner can accumulate predictive knowledge throughout deployment.

Predictive representations and JEPA. Prediction in learned representations is central to LeCun's proposed architecture for autonomous intelligence [18]. CPC [32] and SPR [30] learn representations through future prediction. DINO and DINOv2 [5, 26] provide transferable visual features, while R3M [25] learns manipulation features from human video. BYOL [7] predicts target-network embeddings; Barlow Twins [40] and VICReg [3] use redundancy reduction and statistical regularization to learn non-collapsed representations. I-JEPA [1] predicts target embeddings from image context, V-JEPA [4] extends feature prediction to video, and V-JEPA 2 [2] adds action-conditioned predictions for robotic planning. DINO-WM [41] predicts pretrained patch features and optimizes actions toward goal embeddings. Temporal straightening [36] improves planning geometry, DDP-WM [39] separates primary dynamics from background updates, and LeWorldModel [22] learns predictive embeddings end to end from pixels. SE-JEPA makes deployment experience persistent within this planning framework, consolidating unfamiliar conditions into predictors that can be retrieved on later encounters. We evaluate this mechanism with gradient-based and sampling-based action optimization.

Test-time adaptation. Test-time training [31] and entropy minimization [33] adapt deployed models using unlabeled target data; continual test-time adaptation follows a changing distribution [34]. Control supplies a natural self-supervised signal: every executed action yields an observation that checks the prediction. AdaJEPA [35] uses this signal inside MPC, with adaptation reset between episodes. SE-JEPA builds on this fast loop and retains its experience through slow consolidation, allowing subsequent episodes to start from a predictor informed by earlier interactions.

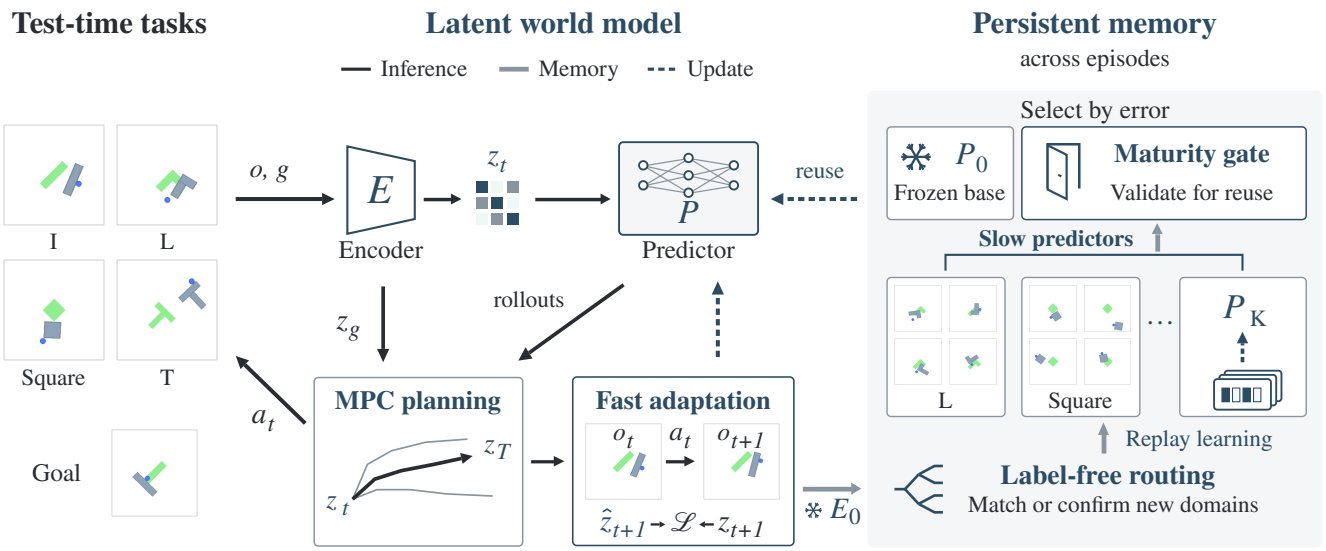


Figure 3. **SE-JEPA architecture.** The fast loop plans in latent space and adapts the predictor from observed transitions. At the end of an episode, the slow system re-encodes the trajectory with the frozen encoder, assigns unfamiliar experience to a self-discovered domain, and trains that domain's slow predictor by replay. Predictors that pass the maturity gate compete with the frozen base at the start of the next episode. Fast parameters reset at every episode; slow predictors and replay persist. Shape names illustrate memory contents and are not supervision.

Continual learning and complementary memory. Complementary learning systems pair rapid hippocampal learning with slow neocortical consolidation [23]. Replay revisits earlier experience [27], GEM constrains updates using episodic memory [21], and EWC protects important parameters [17]. Learning without Forgetting preserves earlier outputs through distillation [19]; progressive networks retain separate task modules [28]. SE-JEPA brings complementary memory into predictive control, where retrieving a model changes both the actions taken and the data collected next. Its label-free routing organizes experience into domains, and its readiness gate evaluates consolidated predictors on episodes they have not yet trained on before using them for planning.

3. SE-JEPA

SE-JEPA turns episode-local corrections into reusable planning knowledge through replay in a fixed latent space, label-free domain discovery, and validation before retrieval (Fig. 3). It keeps the pretrained encoder and predictor (E_0, P_0) as a fixed reference and learns additional predictors from deployment experience. Each discovered domain stores a slow predictor P_{ϕ_k}, a replay buffer $\mathcal{R}_k$, an update-direction prototype, and a validation history.

3.1. Deployment setting and motivation

An agent receives episodes $e = 1, 2, \ldots$, each with an initial observation o_0^e and a goal image g^e. It acts, observes the consequences, and may learn from them. Neither the type nor the onset of a distribution shift is announced; episode boundaries only mark when the scene is reset.

The world model consists of an encoder E_ψ and an action-conditioned predictor P_θ. With history length h, the encoder produces $z_t = E_\psi(o_t)$ and the predictor estimates $\hat{z}_{t+1} = P_\theta(x_t)$ from $x_t = (z_{t-h+1:t}, a_{t-h+1:t})$. The observation embedding includes visual features and proprioception. Rolling P_θ forward under a candidate action sequence, MPC solves

$$a^\star_{t:t+H-1} = \arg\min_{a_{t:t+H-1}} \mathcal{J}\big(\hat{z}_{t+1:t+H},\, E_\psi(g)\big), \qquad (1)$$

executes the first action block, observes the result, and replans. The optimizer is either GD or CEM. SE-JEPA leaves $\mathcal{J}$ and the optimizer unchanged; it changes only the predictor that MPC consults.

Why consolidate experience? Test-time adaptation produces a sequence of fast updates $\Delta_1, \Delta_2, \ldots$, one per episode. The direct inheritance rule accumulates a persistent offset, $s_{e+1} = s_e + w\Delta_e$, and starts the next episode from $\theta_0 + s_{e+1}$. Here $w = 0$ is episodic reset and $w = 1$ keeps every update in full.

On matched targets, carrying every update forward loses 14 percentage points of success relative to episodic reset (Table 1, exact McNemar test, $p = 0.029$). Small inheritance weights yield results closer to reset. Consecutive updates are nearly orthogonal on this stream and on a longer stream

Table 1. **Keeping the adapted weights hurts.** Success on a matched 100-episode PushObj stream (I, square, T, I, square; 20 episodes each). Wins/losses count episodes solved only by the variant / only by reset.

Accumulation w	Success (%) ↑	Wins / losses
0 (episodic reset)	62.0	—
0.003	64.0	7 / 5
0.01	59.0	5 / 8
0.03	54.0	4 / 12
1 (keep all)	48.0	11 / 25

with both planners (Appendix D). Each update fits the actions taken around a particular start and goal. We therefore retain the observed transitions and train a predictor jointly on episodes with compatible dynamics.

3.2. Fast adaptation and replay consolidation

Within an episode, the fast model follows AdaJEPA: after each replan, one Adam step updates the predictor's last block and encoder head using the recent observed transitions. These changes are reset at the next episode. A retrieved slow predictor can supply a better starting point for this adaptation.

To retain experience across episodes, we re-encode the executed trajectory with the frozen encoder E_0. This gives replay a stable coordinate system even as the fast encoder head changes. An accepted trajectory enters its domain's replay, and the slow predictor is trained on sampled transition windows:

$$\mathcal{L}_k(\phi_k) = \mathbb{E}_{(x,z^+)\sim\mathcal{R}_k} \left\| P_{\phi_k}(x) - z^+ \right\|_2^2. \tag{2}$$

Here x contains the latent history and executed actions, and z^+ is the next observation embedding. Sampling windows from earlier episodes fits their transitions jointly with the new experience. A new slow predictor starts from P_0; it learns from replay rather than inheriting the last fast update.

3.3. Discovering predictive domains

The reference model determines which episodes enter memory. We measure its newest-frame prediction error over the first three observed transitions and compare their geometric mean with a calibrated familiarity threshold. Episodes already explained by the reference use it as the base for fast adaptation and do not enter persistent replay. Slow learning is thus concentrated on experience that extends the reference model.

For other episodes, the fast parameter change Δ_e provides a direction fingerprint, $u_e = \Delta_e / \|\Delta_e\|_2$. An episode that retrieved a slow predictor first tries to write to that predictor's domain. Otherwise, cosine similarity to the stored direction prototypes proposes a domain, and a prediction-error check tests compatibility. Unmatched episodes wait in a pending pool. An incoming fingerprint aligned with two pending episodes establishes a new domain with those episodes. This discovers groups of compatible predictive experience without shape or appearance labels.

Domains can also share experience. We merge two domains when each predictor transfers to the other's recent episodes and both improve on the reference. Small immature fragments are absorbed by a mature domain that already predicts them well. Appendix A specifies the confirmation, merging, and absorption rules.

3.4. Validated retrieval for planning

Before training a slow predictor on an incoming episode, we evaluate its open-loop prediction of that episode's executed trajectory τ_e:

$$r_{e,k} = \frac{\mathcal{L}_{\text{roll}}(\phi_k; \tau_e)}{\mathcal{L}_{\text{roll}}(\theta_0; \tau_e)}. \tag{3}$$

This prospective measurement checks whether previous experience transfers to the new trajectory. A high ratio rejects the proposed write. To become eligible for retrieval, a slow predictor must build a validation history and outperform the reference on average by a preset margin (Appendix A).

During the first three replans, the reference and eligible slow predictors are compared on the transitions observed so far:

$$k^\star = \underset{k\in\{0\}\cup\mathcal{K}_{\text{eligible}}}{\arg\min} \mathcal{L}_{\text{pred}}(\phi_k; \mathcal{B}_{e,t}), \qquad \phi_0 = \theta_0. \tag{4}$$

The selected predictor supplies the base for fast adaptation, retaining the episode's current fast offset. Reference-familiar episodes restrict this choice to P_0. Prediction error therefore serves two distinct purposes: prospective validation determines which memories are eligible, and current observations determine which eligible memory to use.

During deployment, replay, slow weights, and domain structure evolve after each episode. Held-out evaluation fixes this memory while retaining retrieval and episode-local adaptation, measuring how the accumulated experience transfers to new targets.

4. Experiments

4.1. Setup

Tasks and backbones. We study shape and appearance shifts with ResNet-based world models trained on task trajectories [15]. In *PushObj*, pretraining covers T, L, Z, and plus shapes. Deployment also introduces I, square, and small tee. *PushT* keeps the T shape and changes its appearance through blur, noise, darkness, or recoloring of the agent, object, or goal. We also test replay in two deformable manipulation tasks, *rope* and *granular*, using DINO-WM with pretrained DINOv2 features [20, 26, 41].

Table 2. **Goal-conditioned planning with CEM.** PushObj and PushT report success (%) and mean replans (failures count as 30); rope and granular report Chamfer distance. PushT Shift covers six shifts; Color covers three recolorings. Rigid-object baselines above the rule use one seed (VIS50 on PushT). Bold compares the matched methods below the rule. $^{\circ}$: offline PushT adapter; ^{T}: T-only training. Italic CD values are published [41]; Frozen uses DINO-WM for the deformable tasks.

	PushObj			PushT visual shifts			Rope	Granular
Method	All ↑	Unseen ↑	Steps ↓	Shift (6) ↑	Color (3) ↑	Steps ↓	CD ↓	CD ↓
DreamerV3° [12]	—	—	—	5.7	0.0	28.91	*2.49*	*1.05*
IRIS$^{\circ}$ [24]	—	—	—	6.0	6.7	28.85	*1.11*	*0.37*
R3M^{T} [25]	18.9	18.0	25.49	21.7	27.3	25.00	*1.13*	*0.95*
DINO CLS^{T}	14.9	10.0	26.77	34.7	35.3	22.38	*0.84*	*0.79*
DINO-WM [41]	62.0	46.0	16.15	57.0	65.3	17.32	0.491	0.301
LeWM [22]	64.0	50.0	16.28	40.3	16.0	21.45	0.786	0.322
DDP-WM [39]	57.7	43.3	17.59	52.7	57.3	18.39	0.677	0.259
Frozen	43.8	36.2	21.08	48.3	33.6	20.10	0.491	0.301
AdaJEPA [35]	70.4	54.4	16.02	60.2	42.2	18.46	0.629	0.304
SE-JEPA	**79.1**	**69.9**	**13.53**	**76.1**	**65.2**	**14.46**	**0.252**	**0.220**

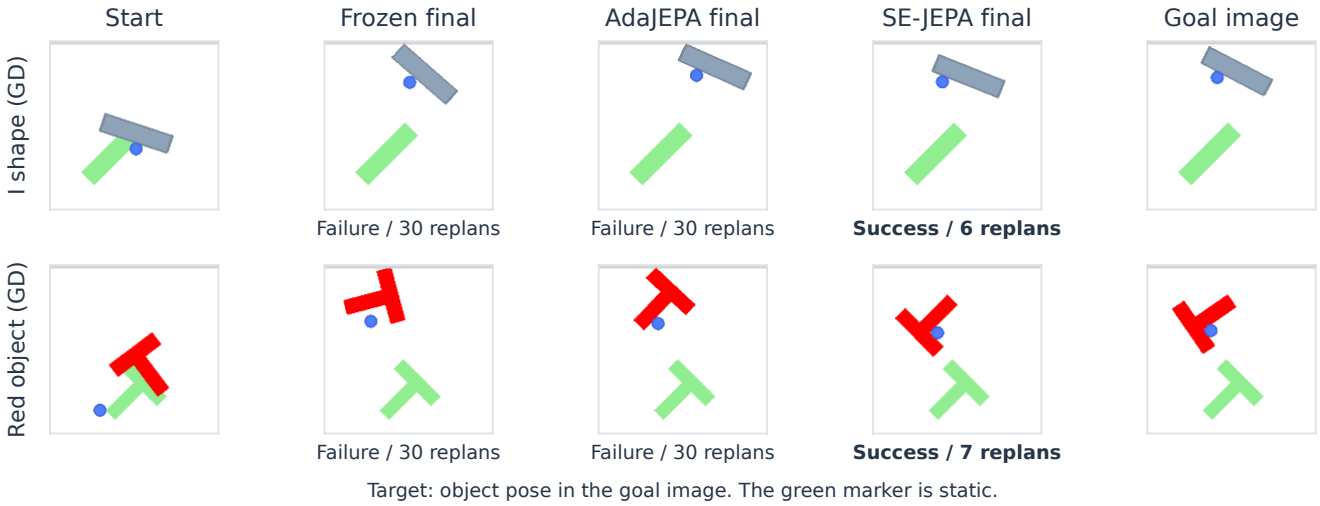


Figure 4. **Persistent memory resolves failures of episodic adaptation.** Selected GD examples on an unseen I shape (top) and red-object shift (bottom). Frozen and AdaJEPA exhaust 30 replans in both cases; SE-JEPA succeeds in six and seven, respectively. Each row shares the same start and goal. The target is the movable object's pose in the goal image; the green marker is static.

Deployment and evaluation. On PushObj and PushT, we run three learning chains per planner. Each chain runs for five rounds with 50 episodes per shape or condition per round and freshly sampled targets. We fix each final memory and test it on held-out targets from three new seeds, keeping retrieval and within-episode adaptation active. The held-out targets come from source episodes absent from the learning streams. On rope and granular, the agent revisits the same 50 instances over four passes, using a single slow predictor per task with fast adaptation disabled. We report Chamfer distance (CD) between the final and goal configurations.

Baselines and planners. On rigid objects, frozen planning and AdaJEPA [35] use the released task-specific checkpoints and the same held-out targets as SE-JEPA. We also compare with DINO-WM, LeWM, DDP-WM, and planners based on DINOv2 CLS and R3M features [22, 25, 26, 39, 41]. We include DreamerV3 [12] and IRIS [24] as generative predictors trained offline on PushT trajectories and used for CEM planning. External rigid-object comparisons use a single test seed. Rigid-object MPC executes five actions per replan, with a budget of 30 replans. Appendix B gives the planner settings and sampling protocol, and Appendix C specifies checkpoints and evaluation groups.

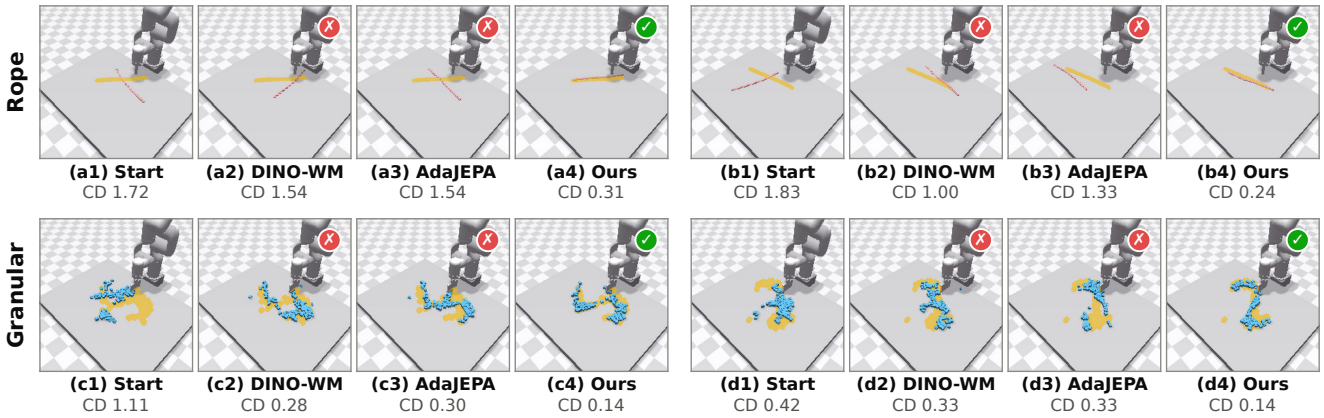


Figure 5. **Consolidated predictions support deformable manipulation.** Real FleX simulator trajectories for rope (top) and granular material (bottom), with two instances per task. Each group shows the start and the three methods' final states after 15 pushes; gold denotes the goal. These third-pass examples illustrate successful slow-only consolidation and were selected where both baselines fail. Frozen DINO-WM frames replay its executed actions. The aggregate fourth-pass results are reported in Table 2.

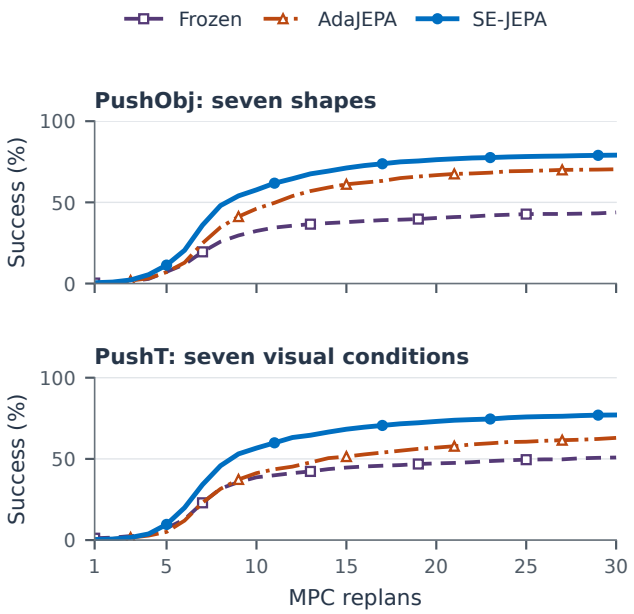


Figure 6. **Higher success within the same interaction budget.** Cumulative held-out success under CEM. Curves use all seven shapes (top) and all seven visual conditions (bottom).

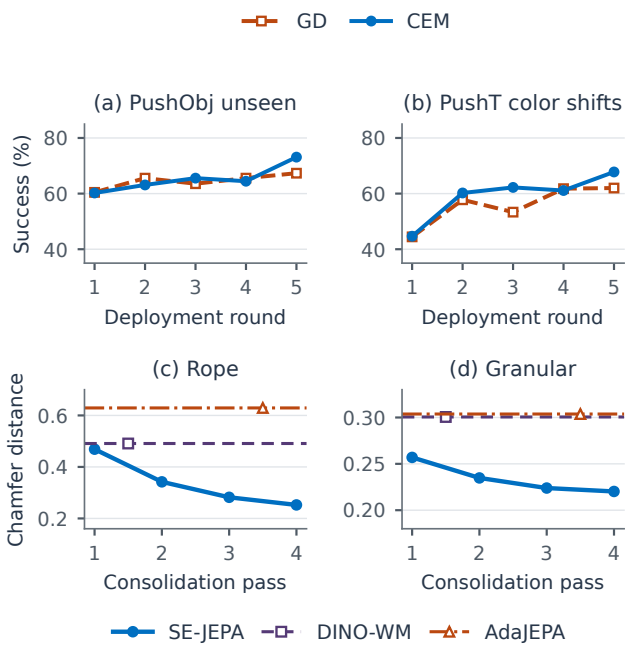


Figure 7. **Deployment experience improves two backbone families.** Top: ResNet-based models, with five rounds of online success on unseen PushObj shapes and three PushT color shifts; curves average three learning chains with targets resampled each round. Bottom: DINO-WM, with four consolidation passes over 50 fixed rope or granular instances; curves show mean CD. The horizontal lines are frozen DINO-WM and AdaJEPA.

4.2. Planning with memory on held-out targets

On shapes absent from pretraining, held-out CEM success is about 15 percentage points higher with SE-JEPA than with AdaJEPA (Table 2). The gain is larger on these shapes than on the shapes seen in pretraining. On PushT, memory helps most when the object or its goal changes color (Appendix D). Both GD and CEM solve more goals with fewer replans on average than episodic adaptation (Appendix B). The CEM success curves separate early in the episode, so the advantage is already visible under a short interaction budget (Fig. 6).

Slow consolidation also improves planning with a different visual backbone. The DINO-WM predictor trained by

replay brings both rope and granular material closer to their target configurations than the frozen DINO-WM predictor (Table 2). Here the planner uses the consolidated predictor directly, allowing us to measure what replay contributes on its own.

4.3. Qualitative results

Figure 4 shows matched starts and goals. On the unseen I shape, SE-JEPA reaches a pose that both baselines miss. In the red-object episode, the baselines leave the T at the wrong orientation, while SE-JEPA turns it into the goal pose. Figure 5 shows the final rope and particle configurations after the executed pushes. With the slow predictor alone, the planner moves the rope toward its target shape and the particles toward the goal distribution. Appendix E gives additional trajectories.

4.4. Learning over deployment

When targets are resampled each round, both GD and CEM finish deployment with higher online success on unseen shapes and recolored scenes than in the first round (Fig. 7, top). These curves track performance as memory grows. Table 2 tests that memory after learning, on independent rigid-object goals.

Revisiting the same rope and granular instances reduces final CD over successive passes (Fig. 7, bottom). To test whether the rope predictor also transfers, we froze it after its first pass and used it on 91 previously unseen instances. On those instances, mean CD was 47% lower than with the pretrained DINO-WM predictor (Appendix B).

5. Conclusion

A push taken to reach one goal can help the planner reach the next. SE-JEPA records the observed transition, replays it alongside compatible experience, and uses the resulting predictor once it improves on the pretrained reference on new trajectories. On rigid objects, this memory lets the agent solve more held-out goals with fewer replans under both GD and CEM. On deformable objects, replay improves planning with a DINO-WM backbone and transfers to new rope instances after a single pass.

Future work. Longer interaction histories could help label-free routing distinguish domains that respond similarly to the first few actions. Late in deployment, lower prediction error does not always translate into more solved goals. Adapting the planning horizon and action search could help the controller put those improvements to use.

LLM Usage Statement

Large language models (OpenAI Codex and Anthropic Claude) were used to draft and revise substantial portions of the manuscript text, and to assist with literature and source checks, LaTeX organization, figure generation, and result aggregation. They also assisted with experimental code, including offline world-model adapters and experiment orchestration. All quantitative results are taken from saved experimental outputs or explicitly cited published sources; language-model output is not used as experimental evidence. The authors take full responsibility for the claims, citations, code, and final verification of the manuscript.

References

[1] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023. 2

[2] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025. 2

[3] Adrien Bardes, Jean Ponce, and Yann LeCun. VICReg: Variance-invariance-covariance regularization for self-supervised learning. In *International Conference on Learning Representations*, 2022. 2

[4] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. Revisiting feature prediction for learning visual representations from video. *arXiv preprint arXiv:2404.08471*, 2024. 2

[5] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *IEEE/CVF International Conference on Computer Vision*, 2021. 2

[6] Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *Advances in Neural Information Processing Systems*, 2018. 2

[7] Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre H. Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Daniel Guo, Mohammad Gheshlaghi Azar, Bilal Piot, Koray Kavukcuoglu, Rémi Munos, and Michal Valko. Bootstrap your own latent: A new approach to self-supervised learning. In *Advances in Neural Information Processing Systems*, 2020. 2

[8] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018. 2

[9] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. In *International Conference on Machine Learning*, pages 2555–2565, 2019. 2

[10] Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2020. 2

[11] Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering atari with discrete world models. In *International Conference on Learning Representations*, 2021.

[12] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse control tasks through world models. *Nature*, 2025. 2, 5

[13] Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations*, 2024. 2

[14] Nicklas A. Hansen, Hao Su, and Xiaolong Wang. Temporal difference learning for model predictive control. In *International Conference on Machine Learning*, pages 8387–8406, 2022. 2

[15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016. 4

[16] Lukasz Kaiser, Mohammad Babaeizadeh, Piotr Milos, Blazej Osinski, Roy H. Campbell, Konrad Czechowski, Dumitru Erhan, Chelsea Finn, Piotr Kozakowski, Sergey Levine, Afroz Mohiuddin, Ryan Sepassi, George Tucker, and Henryk Michalewski. Model-based reinforcement learning for atari. In *International Conference on Learning Representations*, 2020. 2

[17] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017. 3

[18] Yann LeCun. A path towards autonomous machine intelligence. OpenReview position paper, 2022. 2

[19] Zhizhong Li and Derek Hoiem. Learning without forgetting. In *European Conference on Computer Vision*, pages 614–629, 2016. 3

[20] Xingyu Lin, Yufei Wang, Jake Olkin, and David Held. SoftGym: Benchmarking deep reinforcement learning for deformable object manipulation. In *Proceedings of the 2020 Conference on Robot Learning*, pages 432–448, 2021. 4

[21] David Lopez-Paz and Marc'Aurelio Ranzato. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems*, 2017. 3

[22] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026. 1, 2, 5

[23] James L. McClelland, Bruce L. McNaughton, and Randall C. O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419–457, 1995. 1, 3

[24] Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models. In *International Conference on Learning Representations*, 2023. 2, 5

[25] Suraj Nair, Aravind Rajeswaran, Vikash Kumar, Chelsea Finn, and Abhinav Gupta. R3M: A universal visual representation for robot manipulation. In *Proceedings of the 6th Conference on Robot Learning*, pages 892–909, 2023. 2, 5

[26] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jégou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024. 2, 4, 5

[27] David Rolnick, Arun Ahuja, Jonathan Schwarz, Timothy Lillicrap, and Gregory Wayne. Experience replay for continual learning. In *Advances in Neural Information Processing Systems*, 2019. 3

[28] Andrei A. Rusu, Neil C. Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive neural networks. *arXiv preprint arXiv:1606.04671*, 2016. 3

[29] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020. 2

[30] Max Schwarzer, Ankesh Anand, Rishab Goel, R. Devon Hjelm, Aaron Courville, and Philip Bachman. Data-efficient reinforcement learning with self-predictive representations. In *International Conference on Learning Representations*, 2021. 2

[31] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei A. Efros, and Moritz Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *International Conference on Machine Learning*, 2020. 2

[32] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*, 2018. 2

[33] Dequan Wang, Evan Shelhamer, Shaoteng Liu, Bruno Olshausen, and Trevor Darrell. Tent: Fully test-time adaptation by entropy minimization. In *International Conference on Learning Representations*, 2021. 2

[34] Qin Wang, Olga Fink, Luc Van Gool, and Dengxin Dai. Continual test-time domain adaptation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022. 2

[35] Ying Wang, Oumayma Bounou, Yann LeCun, and Mengye Ren. AdaJEPA: An adaptive latent world model. *arXiv preprint arXiv:2606.32026*, 2026. 1, 2, 5, 10

[36] Ying Wang, Oumayma Bounou, Gaoyue Zhou, Randall Balestriero, Tim G. J. Rudner, Yann LeCun, and Mengye Ren. Temporal straightening for latent planning. In *International Conference on Machine Learning*, 2026. 1, 2

[37] Manuel Watter, Jost Tobias Springenberg, Joschka Boedecker, and Martin Riedmiller. Embed to control: A locally linear latent dynamics model for control from raw images. In *Advances in Neural Information Processing Systems*, 2015. 2

[38] Weirui Ye, Shaohuai Liu, Thanard Kurutach, Pieter Abbeel, and Yang Gao. Mastering atari games with limited data. In *Advances in Neural Information Processing Systems*, 2021. 2

[39] Shicheng Yin, Kaixuan Yin, Weixing Chen, Yang Liu, Guanbin Li, and Liang Lin. DDP-WM: Disentangled dynamics prediction for efficient world models. *arXiv preprint arXiv:2602.01780*, 2026. 2, 5

[40] Jure Zbontar, Li Jing, Ishan Misra, Yann LeCun, and Stephane Deny. Barlow twins: Self-supervised learning via redundancy reduction. In *International Conference on Machine Learning*, pages 12310–12320, 2021. 2

[41] Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning. In *International Conference on Machine Learning*, pages 79115–79135, 2025. 1, 2, 4, 5, 12

Supplementary Material

A. Implementation and Memory Management

Compute environment. Model training and evaluation were conducted on systems equipped with 8× NVIDIA GeForce RTX 5090 GPUs and 8× NVIDIA A100 GPUs.

A.1. Fast and slow learning

The label-free rigid-object configuration builds on the episodic adaptation loop of AdaJEPA [35]. At every replan, the most recent observed transitions provide a self-supervised next-latent loss. The predictor's last block and the encoder's designated projection/head parameters are adapted with one Adam step. The episode-local predictor and encoder changes are reset at the next episode. A retrieved slow predictor supplies the predictor initialization; it does not replace the frozen reference encoder.

Persistent replay is re-encoded by the reference encoder. Each slow learner is a full predictor copy, initialized from the pretrained predictor rather than from the final fast parameters of its founding episode. For each slow optimization step, up to eight stored episodes are sampled with replacement; each contributes a random window of up to six latent frames. The prediction objective averages one-step observation-feature errors over the available history windows. Action features condition prediction but are not reconstruction targets. Standard writes use 15 Adam steps. Founding a learner after n stored episodes uses $15n$ steps to process the accumulated evidence. Merging likewise adds replay updates for absorbed experience.

Replay stores compact latent trajectories rather than images. The pending pool is bounded, and merging and absorption remove redundant predictor copies as compatible domains are recognized.

A.2. Order of decisions

Algorithm 1 summarizes a learning episode. Familiarity uses the geometric mean of newest-frame prediction errors on up to the first three observed transitions. The threshold is calibrated once on the shape backbone and transfers unchanged to the visual experiments.

The selected predictor's error is evaluated before it learns from the current episode, so maturity reflects transfer to data the predictor has not fitted. In read-only evaluation, the episode-end write, validation-history update, slow training, merging, and absorption steps are all disabled.

A.3. Fingerprints and delayed domain founding

A fingerprint concatenates the adapted predictor and encoder-head changes and is normalized to unit length. Domain prototypes accumulate normalized fingerprints from episodes planned from the reference; episodes starting from a slow predictor do not redefine that reference-based prototype. If an episode has already read a learner, it preferentially writes to that learner's domain unless the prospective error ratio rejects the write. Otherwise, it tries compatible domain directions in descending cosine order.

An unassigned episode enters a pending pool. It confirms a new domain when at least two pending fingerprints align with its own above the new-domain threshold. These members need not be pairwise aligned with one another; the incoming fingerprint is the common reference for this confirmation. The members and the new episode are transferred together to the new replay. No ground-truth shape or appearance category participates in this procedure.

Table 3. Label-free configuration used for PushObj and PushT. The seven-round stream in Appendix D uses a slow learning rate of 2×10^{-4}.

Setting	Value
Fast predictor learning rate	5×10^{-4}
Fast encoder-head learning rate	10^{-5}
Fast steps per replan	1
Recent adaptation buffer	5 transitions
Slow learning rate	10^{-4}
Slow steps per standard write	15
Slow batch size (episodes)	8
Slow sampled window (frames)	up to 6
Familiarity threshold τ_f	0.0146
Existing-domain cosine threshold	0.10
New-domain cosine threshold	0.05
Episodes to confirm a domain	3
Pending-pool capacity	40
Minimum episodes to train a learner	3
Retrieval window	first 3 replans
Maturity history length m	3
Maturity error-ratio threshold τ_m	0.9
Write rejection ratio	1.2
Merge evidence per direction	2 episodes
Merge relative tolerance	0.5
Fragment absorption interval	25 episodes
Absorption error-ratio threshold	0.8
Fragment-size limit	< 6 episodes

A.4. Merging and fragment absorption

For a new unfamiliar episode, every existing trained learner is evaluated before training. The resulting ratios provide cross-domain evidence. Let $G_{A\to B}$ be the geometric mean error ratio of learner B on recent episodes assigned to A, and $G_{A\to A}$ the corresponding ratio for learner A. Domains merge only if both directions satisfy

$$G_{A\to B} \le \min\{1, (1+\epsilon_m) G_{A\to A}\}, \tag{5}$$

Algorithm 1 One deployment-learning episode of SE-JEPA

Require: Frozen (E_0, P_0), memory $\mathcal{M}$, initial observation o, goal g
Reset fast parameters to (E_0, P_0)
Initialize recent buffer $\mathcal{B}$ and episode trajectory τ
while budget remains and task is not complete **do**
    Optimize an action sequence with the current fast model
    Execute its first block; observe the next state
    Append real transitions to $\mathcal{B}$ and τ
    if within the early retrieval window **then**
        Form candidates: P_0 and mature slow predictors
        Restrict to P_0 if the observed episode is familiar
        Select the smallest observed prediction error
        Load the selected predictor, retaining the fast offset
    end if
    Adapt the fast model on recent real transitions
end while
Encode τ with frozen E_0
Measure pre-update rollout errors of existing predictors
if episode is unfamiliar **then**
    Propose a domain using retrieval or an update fingerprint
    Reject an incompatible write, or retain pending evidence
    if a domain is assigned or newly confirmed **then**
        Record prospective validation when a learner exists
        Append latent experience; update its slow predictor
        Check bidirectional merges and periodic absorption
    end if
end if
Discard episode-local fast parameters

with the symmetric condition for $B \to A$. Each direction requires two jointly evaluated episodes in the primary configuration. The larger domain retains its learner and incorporates the union of stored experience. Merged identifiers are not reused.

Every 25 episodes, small immature domains can be absorbed into a mature predictor that explains their experience with an error ratio at most 0.8. The primary configuration limits absorption to fragments with fewer than six episodes and fewer than three maturity measurements. Pending-pool absorption and the additional absolute-error gate are disabled in the shape and visual experiments.

B. Evaluation Protocols

B.1. Shape shifts

The shape learning stream has five deployment rounds over seven shapes, with 50 episodes per shape in every round. The held-out benchmark uses target seeds 2000, 2100, and 2200, each with 350 episodes. I, square, and small tee are unseen in pretraining; T, L, Z, and plus are seen.

Each planner has three learned memory states. Every state is evaluated on all three held-out seeds; for each seed we average over episodes and states, and standard deviations are computed over the three seed-level means. The CEM states are S7t3, S7t3b, and S7t3s700, where S7t3b is a rerun of S7t3 that replaces a run in which no learner matured. Frozen and AdaJEPA use the same three held-out seeds; the external baselines use seed 2000.

B.2. Visual shifts

The paired visual evaluation contains the original T condition, blur, salt-and-pepper noise, darkness, red agent, red object, and red goal. Three five-round learning chains resample targets every round, with 350 episodes per round. Held-out testing uses three target seeds and 50 episodes per condition. Direct checks of source-episode identifiers confirm that held-out sources never occur in the learning streams and that the three test seeds use disjoint sources.

The color-learning curve in Fig. 7 aggregates red-agent, red-object, and red-goal episodes. For each round and learning chain, it sums successes over the three conditions and divides by 150 episodes, and plots the mean of the three chain-level rates. The archived statistics include their sample standard deviation. The three conditions share the same 50 source targets within a chain and round.

The external visual baselines use the established VIS50 collection with planning seed 100. The external comparisons use its six shifted conditions, with 50 paired initial/goal states per condition.

B.3. Success and interaction counts

For rigid objects, GD uses 100 optimization steps; CEM uses 10 iterations with 200 samples and 30 elites per iteration. Rigid-object success follows the environment's goal test: object-position error below 20 simulator units and angular error below $\pi/9$. Each executed model action contains five physical actions, and an episode has at most 30 MPC replans. For episode i with success indicator S_i and first successful replan T_i,

$$\overline{T} = \frac{1}{N}\sum_{i=1}^{N}\left[\, S_i T_i + (1 - S_i)\, T_{\max}\right], \qquad T_{\max} = 30. \quad (6)$$

Equivalently, $\overline{T} = T_{\max} - \sum_{t=1}^{T_{\max}-1} F(t)$, where $F(t)$ is the fraction of episodes solved by replan t; the cumulative curves in Fig. 6 visualize F.

B.4. Deformable manipulation

Rope and granular each use one slow predictor, a DINO-WM backbone, and 50 instances revisited over four passes, with the same targets, order, and planning seeds in every pass. The slow-only variant plans from the slow predictor without fast within-episode updates. CEM uses 300 samples, 30 elites, and 10 iterations, with three MPC iterations of five pushes each. The unseen-rope evaluation freezes the predictor saved after the first pass and tests it on 91 instances absent from the learning stream, against the frozen reference on the same instances. Two frozen granular episodes with unstable batched particle simulation were rerun individually with identical settings and seeds.

C. Baseline Checkpoints and Comparisons

Predictive world models. The comparisons focus on latent world models used for visual planning: DINO-WM, LeWM, and DDP-WM, with DINO CLS and R3M as representation baselines. Frozen and AdaJEPA use released task-specific checkpoints. For PushObj and PushT, SE-JEPA uses ResNet-based world models trained directly on task trajectories. Its PushObj backbone is trained for three epochs on 16,000 trajectories covering T, L, Z, and plus. The additional DINO-WM shape model uses the same training trajectory count and epoch budget, with 4,000 validation trajectories and batch size 32. DINO CLS and R3M use existing T-only checkpoints; LeWM and DDP-WM use their corresponding checkpoint adapters.

Offline generative baselines. For PushT we use offline-trained DreamerV3 and IRIS predictors for CEM planning and evaluate them on the same VIS50 targets as DINO-WM. Their published PushObj results use two unseen shapes and a goal test on both the agent and the object [41]; those scores do not measure our three-unseen-shape metric. We leave their PushObj cells unreported.

Evaluation groups. The paired Frozen/AdaJEPA/SE-JEPA comparison uses the independent held-out target sets in Appendix B. External visual baselines use VIS50, with 50 initial/goal pairs in each of six shifted conditions and planning seed 100. The table separates these evaluation groups. Within each group, methods receive the same targets and planning budget. PushT Shift averages blur, noise, dark, red agent, red object, and red goal; Color averages the last three. The original condition is included in the seven-condition tables and curves, but excluded from Shift.

Deformable results. The rope and granular experiments use DINO-WM with pretrained DINOv2 patch features as the frozen backbone. This complements the task-trained ResNet setting used for rigid-object manipulation. AdaJEPA adapts within each episode, whereas SE-JEPA uses a consolidated slow predictor. Italic CD entries for DreamerV3, IRIS, R3M, and DINO CLS come from the ten-instance evaluation of Zhou et al. [41]. Our frozen DINO-WM reference uses the same 50 instances as our deformable adaptation runs and appears in both the DINO-WM and Frozen rows. The repeated-instance and unseen-rope protocols are specified in Appendix B.

D. Additional Results

D.1. All object shapes

Table 5 reports every held-out shape. SE-JEPA gives the largest gains on the unseen shapes: with GD, I rises from 44.7% (AdaJEPA) to 77.6% and square from 46.7% to 61.1%. It also leads on the seen shapes in aggregate.

D.2. All visual conditions

Table 6 and Fig. 8 break the paired visual results down by condition. SE-JEPA improves on all seven conditions with both planners. The largest gains, more than 25 points, appear under red-object and red-goal recoloring, where episodic adaptation has to rediscover the new appearance in every episode.

D.3. Planning efficiency

Figure 6 plots cumulative held-out success against the number of MPC replans. SE-JEPA pulls ahead within the first four replans and keeps the lead for the rest of the budget, showing how the advantage develops before the final 30-replan evaluation.

D.4. Deformable manipulation

Figure 7 shows the four-pass learning curves in the main paper. Table 7 adds standard errors to the deformable results of Table 2. Table 8 follows consolidation pass by pass at two slow learning rates. With 10^{-4}, CD decreases at every pass on both tasks. With 2×10^{-4}, rope improves faster in the early passes and both rates reach a similar level after four passes, so consolidation is robust to the learning rate.

D.5. A seven-round shape stream

A second PushObj stream runs seven rounds with a slow learning rate of 2×10^{-4}, one chain per planner. Each round has 50 episodes per shape, drawn without replacement and excluding the held-out sources, for 2,450 episodes per planner. Online success rises from 70.6% in the first round to 78.6% in the seventh with GD, and from 72.3% to 78.3% with CEM.

The episode logs record the fast-update fingerprint of every episode. Figure 9 shows the cosine between consecutive fingerprints: the mean is 0.018 for both planners,

Table 4. Both planners on the paired held-out sets. Shape All and Unseen use held-out PushObj targets. Visual All averages seven conditions and Shift six; visual Steps averages all seven conditions.

		PushObj			PushT		
Planner	Method	All ↑	Unseen ↑	Steps ↓	All ↑	Shift ↑	Steps ↓
GD	Frozen	42.48	32.44	20.02	44.10	41.56	19.94
GD	AdaJEPA	72.19	55.33	14.52	58.67	54.89	17.96
GD	SE-JEPA	81.59	72.44	11.30	75.68	73.63	13.21
CEM	Frozen	43.81	36.22	21.08	50.86	48.33	19.56
CEM	AdaJEPA	70.38	54.44	16.02	62.95	60.22	17.87
CEM	SE-JEPA	79.08	69.85	13.53	77.05	76.11	14.15

Table 5. Held-out success (%) per PushObj shape. I, square, and small tee are unseen in pretraining. All averages the seven shapes.

Planner	Method	I	L	Square	Z	Small tee	Plus	T	All
CEM	Frozen	32.7	52.7	23.3	41.3	52.7	43.3	60.7	43.8
CEM	AdaJEPA	49.3	82.0	44.7	82.0	69.3	79.3	86.0	70.4
CEM	SE-JEPA	72.7	84.2	58.4	86.2	78.4	85.6	88.0	79.1
GD	Frozen	24.0	52.7	28.0	40.0	45.3	48.7	58.7	42.5
GD	AdaJEPA	44.7	90.0	46.7	82.0	74.7	82.0	85.3	72.2
GD	SE-JEPA	77.6	88.4	61.1	88.0	78.7	87.6	89.8	81.6

and most of the mass lies near zero. These measurements support consolidating transition data rather than accumulating episode-specific parameter updates (Sec. 3.1). Over the stream, SE-JEPA writes 622 (GD) and 663 (CEM) episodes to memory, 25.4% and 27.1% of all episodes, and finishes 630 and 666 episodes with a slow predictor.

D.6. What SE-JEPA chooses to learn

SE-JEPA learns selectively. In a seven-round PushObj stream of 2,450 episodes per planner, it writes only 25% (GD) to 27% (CEM) of episodes to memory; the rest are familiar to the pretrained model, still pending, or rejected by validation. Retrieval is just as targeted: slow predictors plan about one episode in four, and the reference handles the rest.

Selective writing leaves familiar dynamics to the pretrained model, which SE-JEPA never overwrites. On the four pretraining shapes, SE-JEPA reaches 86.0% held-out success with CEM and 88.4% with GD, against 82.3% and 84.8% for AdaJEPA. This preserves a useful pretrained reference while allowing specialized predictors to add capacity for recurring shifts.

E. Qualitative Examples and Reproducibility

E.1. Full trajectories

Figure 4 selects an unseen-I and a red-object episode where both baselines fail and SE-JEPA succeeds under GD. These illustrate changes in planning behavior; aggregate success is evaluated on the complete held-out sets. The source package supplies the first held-out target of test seed 2000 for each condition, and the displayed conditions are chosen by outcome. All methods share identical initial and goal observations. The panels use a consistent crop size to enlarge the movable objects, with every colored foreground element preserved.

Figures 10 and 11 provide additional CEM examples on the first held-out target of the same test seed: an unseen square and a red-agent shift. They show the complete trajectories and each method's prediction loss. These are separate from the GD examples in the main paper.

Each loss curve is measured in the latent space of its own model, so it describes how prediction error evolves within a method over the episode. Task success, judged in the shared simulator state, is the common yardstick across methods.

E.2. Reproducibility

Tables and plots use the bundled reporting snapshot and the archived Drive experiment packages. The revised trajectory panels are rendered directly from the packages' simulator frames; the asset manifest records their source files and hashes. Success and Steps are computed from the final episode files and their run configurations. Offline world-model checkpoints record the upstream code revision, the epoch budget, the model configuration, and hashes of the adapter sources. Code, configurations, and learned memory states will be released.

Table 6. Held-out visual success (%) per condition on the paired targets. All averages seven conditions; Shift excludes Original.

Planner	Method	Original	Blur	Noise	Dark	Red agent	Red object	Red goal	All	Shift
CEM	Frozen	66.0	56.0	62.7	70.7	50.0	14.0	36.7	50.9	48.3
CEM	AdaJEPA	79.3	78.0	76.7	80.0	71.3	14.0	41.3	63.0	60.2
CEM	SE-JEPA	82.7	86.4	86.7	88.0	79.6	47.1	68.9	77.0	76.1
GD	Frozen	59.3	48.7	55.3	58.7	50.7	6.7	29.3	44.1	41.6
GD	AdaJEPA	81.3	77.3	73.3	73.3	58.0	10.7	36.7	58.7	54.9
GD	SE-JEPA	88.0	88.7	84.9	87.3	74.7	44.4	61.8	75.7	73.6

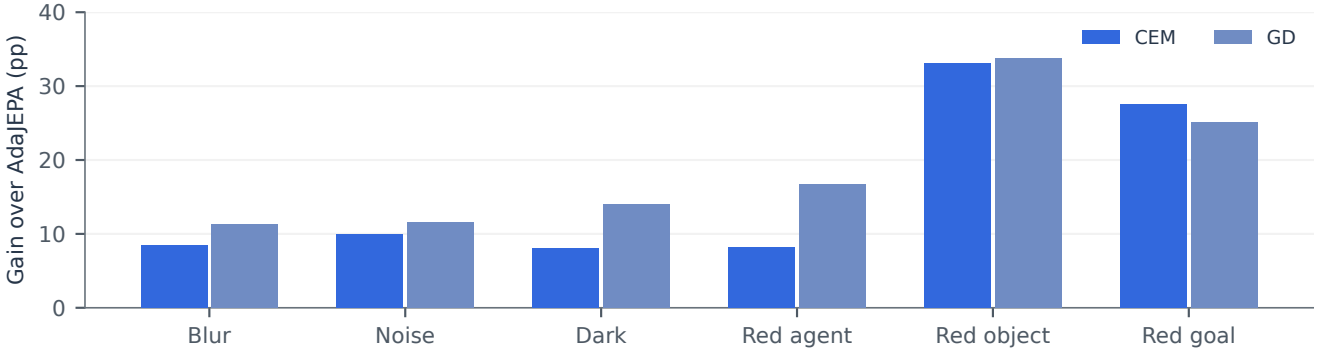


Figure 8. Held-out gain of SE-JEPA over AdaJEPA for each visual shift, in percentage points.

Table 7. Deformable manipulation: mean CD $\pm$ standard error over 50 instances. SE-JEPA uses its slow predictor after four passes.

Method	Rope CD $\downarrow$	Granular CD $\downarrow$
DINO-WM	0.491 ± 0.059	0.301 ± 0.020
LeWM	0.786 ± 0.112	0.322 ± 0.018
DDP-WM	0.677 ± 0.086	0.259 ± 0.012
AdaJEPA	0.629 ± 0.096	0.304 ± 0.019
SE-JEPA	$\mathbf{0.252 \pm 0.045}$	$\mathbf{0.220 \pm 0.012}$

Table 8. CD of SE-JEPA after each consolidation pass at two slow learning rates (mean $\pm$ standard error, 50 instances).

	lr 10^{-4}		lr 2×10^{-4}	
Pass	Rope	Granular	Rope	Granular
1	0.469 ± 0.100	0.257 ± 0.014	0.416 ± 0.070	0.260 ± 0.012
2	0.342 ± 0.054	0.235 ± 0.012	0.286 ± 0.050	0.241 ± 0.012
3	0.282 ± 0.033	0.224 ± 0.010	0.283 ± 0.037	0.247 ± 0.012
4	0.252 ± 0.045	0.220 ± 0.012	0.243 ± 0.033	0.243 ± 0.013

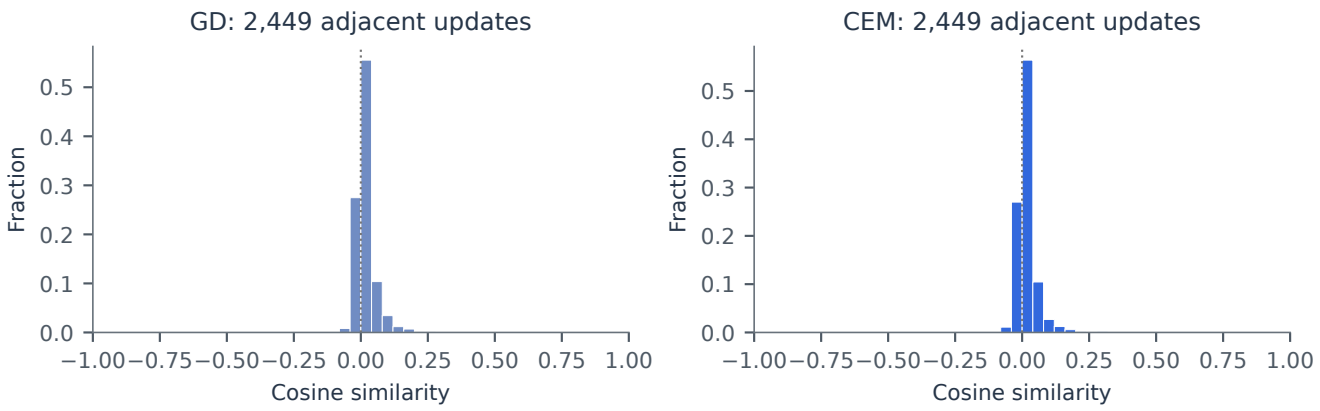


Figure 9. Cosine similarity between consecutive fast-update fingerprints in the seven-round shape stream (2,449 pairs per planner).

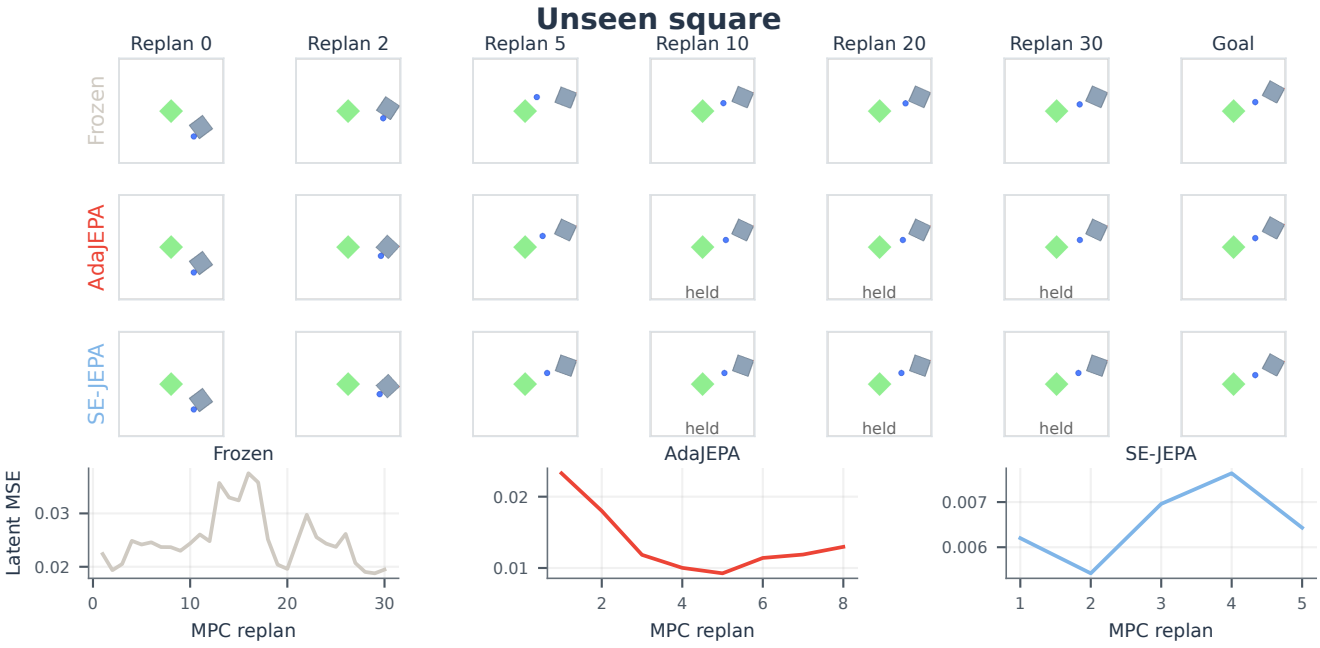


Figure 10. Full unseen-square example with intermediate frames and prediction-loss traces. Frames marked held repeat the terminal state after success. SE-JEPA reaches the goal at replan five, AdaJEPA at replan eight, and the frozen model does not succeed within the budget.

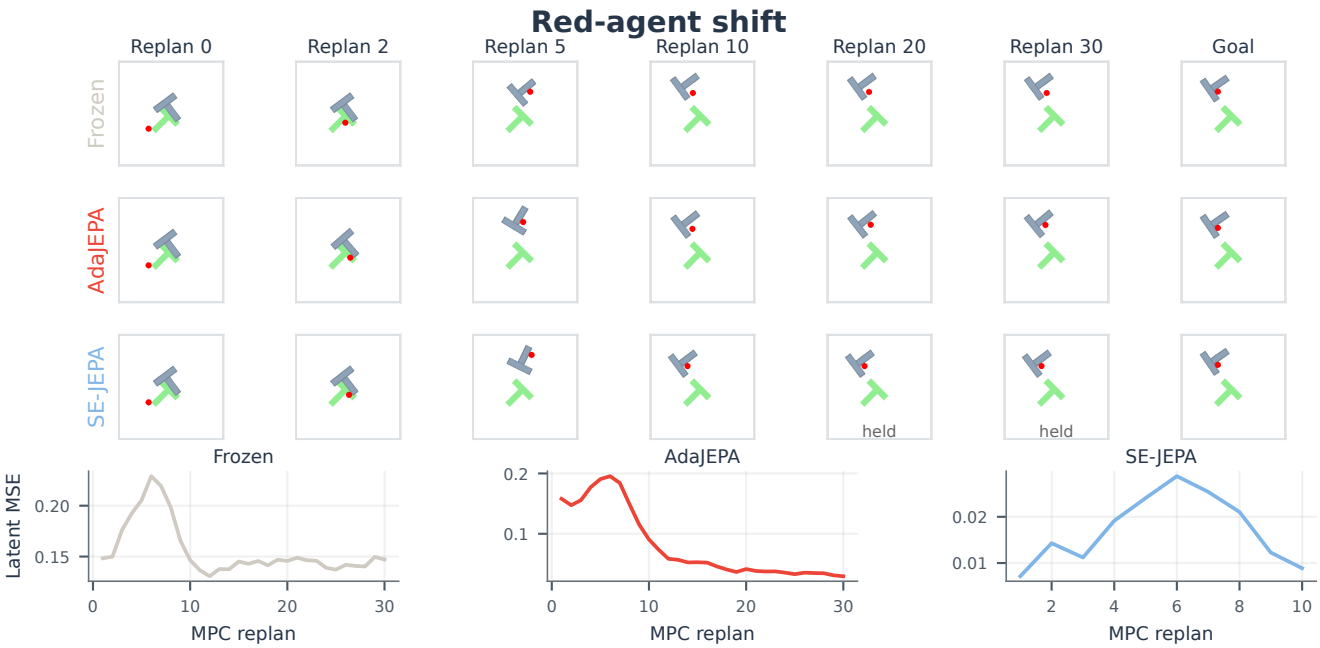


Figure 11. Full red-agent example, replayed in the original simulator from the executed actions. SE-JEPA succeeds at replan ten; Frozen and AdaJEPA reach the 30-replan limit.